

MEDHAVA

The master specification, measured

945 line items across 31 sections. Every one of them is from the specification as it was written, in its order and its words. Nothing was added to lengthen the list and nothing dropped to shorten it — the gate counts the items and refuses a stated total that disagrees with the list.

The three numbers

Covered	200	21%
Uncovered	660	70%
Not possible from here	85	9%

What "covered" means, and what it does not. It means an app exists for that line *and* has reached a rung above SPECIFIED. It does **not** mean the line is finished, and it does not mean the app runs on a real database — three of them do. The single most likely way to misread this sheet is to add up the covered column and treat it as progress. The state column below is the honest breakdown:

Where this product stands on a line	Count
An app that stands above SPECIFIED	203
An app that is SPECIFIED — written down, not built	665
Nothing in the register maps to it at all	77

Overall score **1.4/5**, maturity **Level 3 — Prototype**.

What "not possible from here" means

85 lines cannot be closed by writing code, and that is not a way of saying they are hard. Each needs something a repository cannot hold: a telephony carrier, a payment licence, a bank's credentials, a government portal's registration, an app-store account, live media infrastructure, a deployed host, or an audited certification. Every one of those lines names its own reason in the sheet.

They are counted separately precisely so they do not sit in the uncovered column looking like work somebody forgot to schedule.

The 660 uncovered lines are two different things

660 of them already have an app in this product's own register — the design names the capability, and the app sits at SPECIFIED or has not yet been taken above it. Those cannot be "added"; they are added already and unbuilt, which is a schedule problem rather than a design gap.

0 are genuinely absent — no app in the register maps to them at all. Those are in `brand/site/backlog.js`, and they are not 0 separate pieces of work. They are 12 capabilities:

Lines	Capability	Would live in
0	Developer platform — API, webhooks, functions, a builder	module 23 Developer Platform
0	Agile delivery — backlog, sprints, boards, burndown	module 24 Agile & Sprints
0	Meetings — scheduling, participants, polls, the record of a call	module 25 Meetings
0	Calendar — events, invitations, availability, shared calendars	module 26 Calendar
0	Mail handling — folders, filters, rules, signatures, a shared inbox	module 27 Mail
0	Document applications — writing, presenting, notes, track changes	module 28 Documents
0	Learning — courses, lessons, assessments, progress	module 29 Learning
0	Integration platform — triggers, actions, custom connectors	module 30 Integration Platform
0	Identity and IT — MFA, password management, service management	module 31 Identity & IT
0	Data pipelines, enrichment and a warehouse	module 21 Dashboard & BI
0	Sales territories	module 04 CRM
0	Seven small independent lines	module 01 Platform

They are a backlog and not registry rows, deliberately. Entering them as apps would take the app count from 165 to 165, every new row at the lowest rung — so the tested ratio and the score above would both fall without one line being built. A number that gets worse because the denominator grew is not a measurement of anything.

What this sheet does not measure, said before anybody looks for it

There is no column here for what any other company does against these lines, and that is deliberate rather than an omission. Two reasons, and the first is the honest one:

It was never measurable at this resolution. A sourced claim needs an address and the day somebody read it. Nobody holds ~900 of those about another company's software, and filling the column from recollection would make the sheet look complete while making it worthless — which is the exact failure every gate in this repository exists to stop.

And it answers a question this document was not asked. The question was: of everything in the specification, how much does this product cover. That is measured below, 945 times, and every verdict is resolved from the requirements registry at generation time rather than stored in a row that could flatter itself.

Section by section

#	Section	Items	Covered	Uncovered	Not possible
1	Platform architecture	49	7	38	4
2	Master application hub	100	23	67	10
3	CRM	61	26	34	1
4	Marketing	78	2	64	12
5	Customer support / helpdesk	37	23	11	3
6	Finance and accounting	57	10	44	3
7	Expense management	17	0	15	2
8	Billing and subscriptions	18	1	16	1
9	Procurement	23	15	8	0
10	Inventory	37	10	27	0
11	E-commerce	42	8	29	5
12	POS	20	11	6	3
13	HR	45	3	42	0
14	Recruitment	16	2	12	2
15	Payroll	16	0	16	0
16	Project management	21	4	17	0
17	Agile development	14	0	14	0
18	Document management	28	12	13	3
19	Email	20	0	10	10
20	Calendar	11	0	10	1
21	Team chat	17	2	12	3
22	Video meetings	15	0	8	7
23	Shared team inbox	11	5	4	2
24	Low-code app builder	19	3	14	2
25	Workflow automation	41	2	36	3
26	Integration platform	14	0	12	2

#	Section	Items	Covered	Uncovered	Not possible
27	Data preparation	15	0	14	1
28	Business analytics	44	21	21	2
29	AI platform	29	2	26	1
30	AI agents	18	1	16	1
31	Contract management	12	7	4	1

The full sheet

Every line, with what it maps to and why, is in `MASTER_SPEC_COVERAGE.xlsx` beside this document — one row per item, filterable by the covered / uncovered / not-possible columns. It is generated from the same register as the figures above, so the two cannot disagree.

The mapping is a judgement and no gate can check a judgement. That "Lead scoring" was set against CRM & Customer 360 is a call somebody made. What is checked is that every app named is real and that its rung is resolved from the gated registry rather than typed. Each row prints the app it was mapped to for exactly that reason — so any single one of the 945 calls can be disagreed with.

To check the whole thing yourself, from the repository:

```
node brand/site/checkmasterspec.js --summary
```

Every technical word above, in plain language

5 words. Every technical term this document uses, in plain language, with an everyday comparison. Nothing here assumes you already know any of them.

platform

One piece of software that many separate businesses use at the same time, each seeing only its own information.

Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.

module

One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together.

Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.

database

Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.

Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.

row

One single record — one customer, one order, one payment.

Register mein ek line. Ek line matlab ek entry.

API

The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer.

Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.

Medhava · One business operating system · Any industry